**Sistemas Distribuidos (1959). Grupo: 2**
Alumno: Rodríguez Jaramillo Alejandro
UNAM / FI / DIE
Semestre 2027-1

# S08: Que la transferencia no deje dinero en el aire

## Parte 0 · Ver el problema antes de arreglarlo

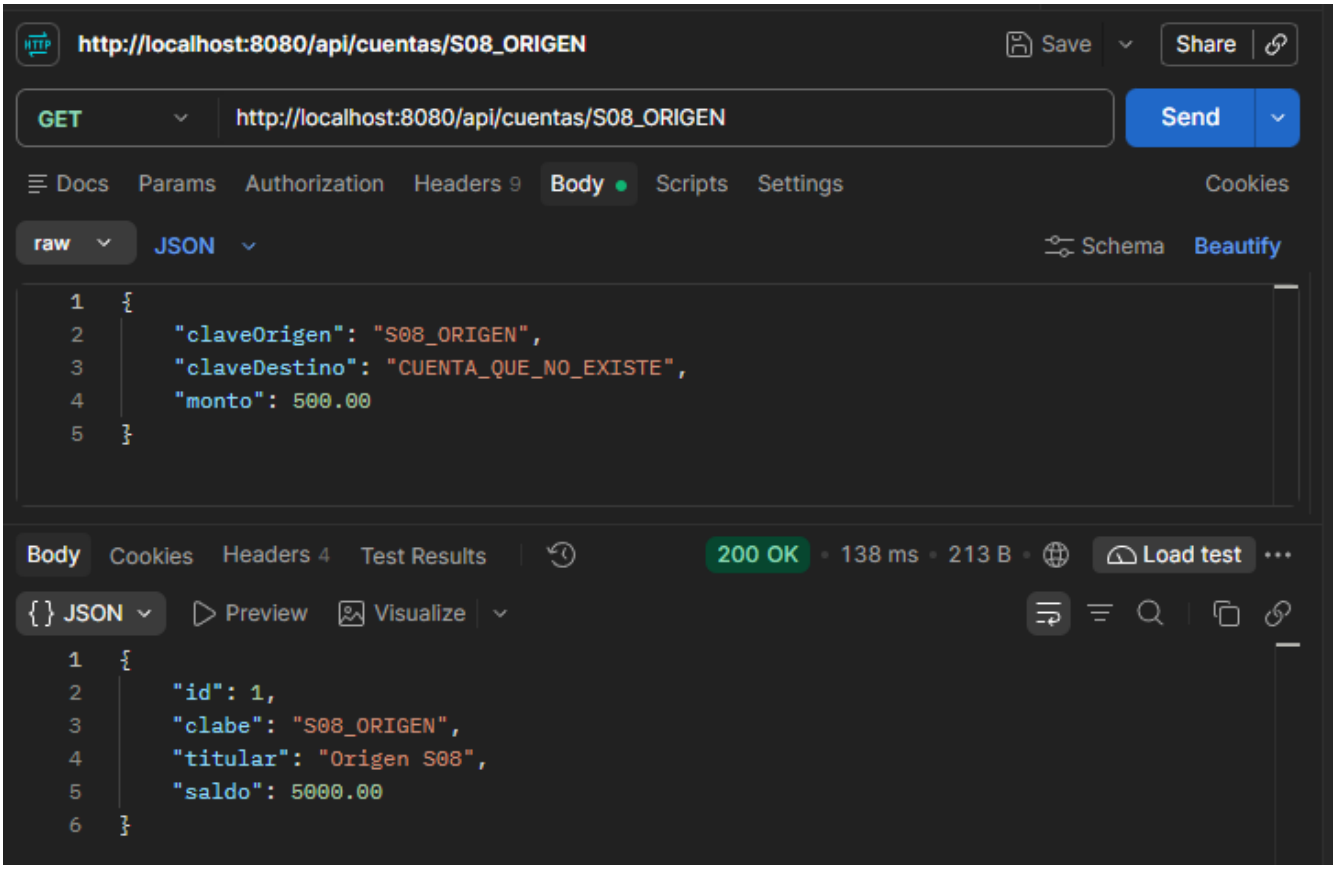



Figure 1: Saldo inicial de la cuenta origen ($5000.00).

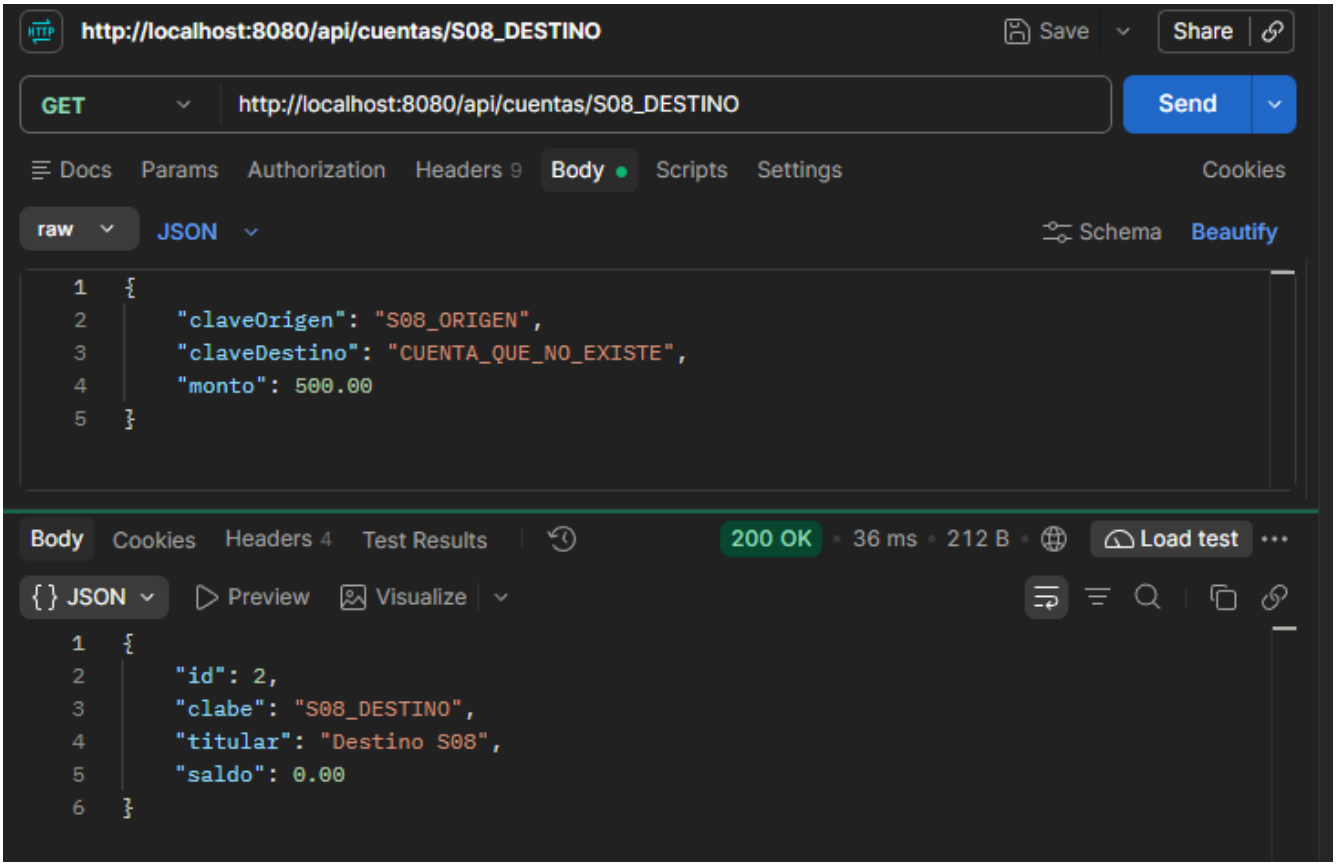



Figure 2: Saldo inicial de la cuenta destino ($0.00).

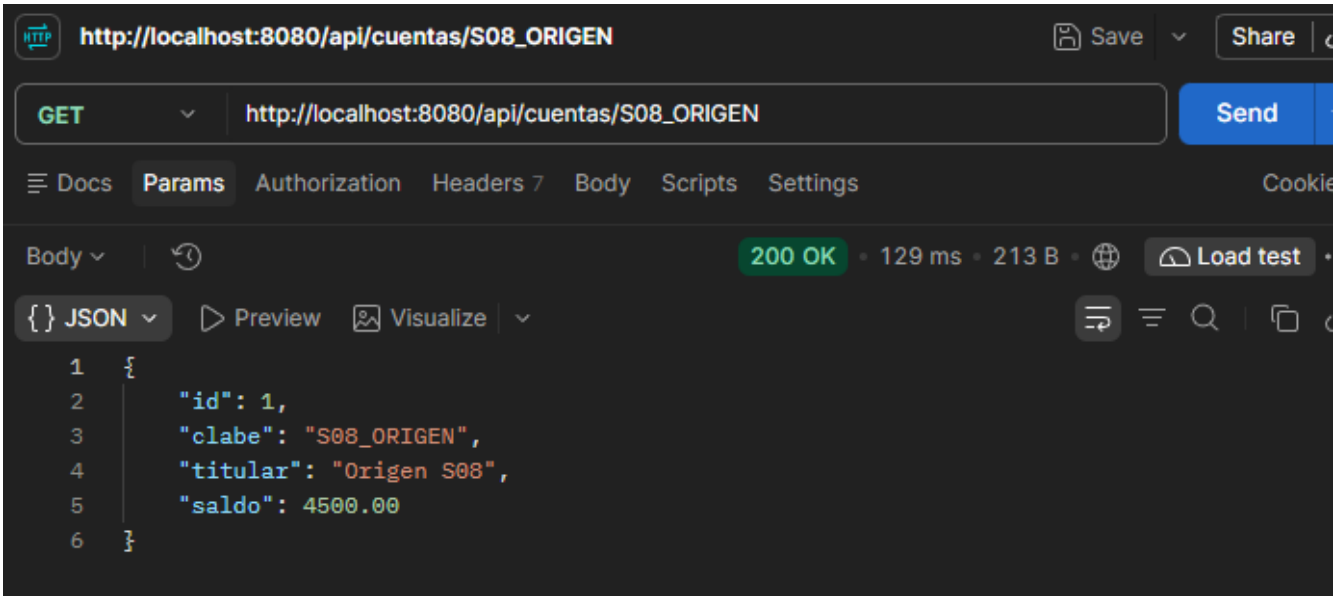



Figure 3: Saldo de la cuenta origen tras el fallo no controlado ($4500.00).

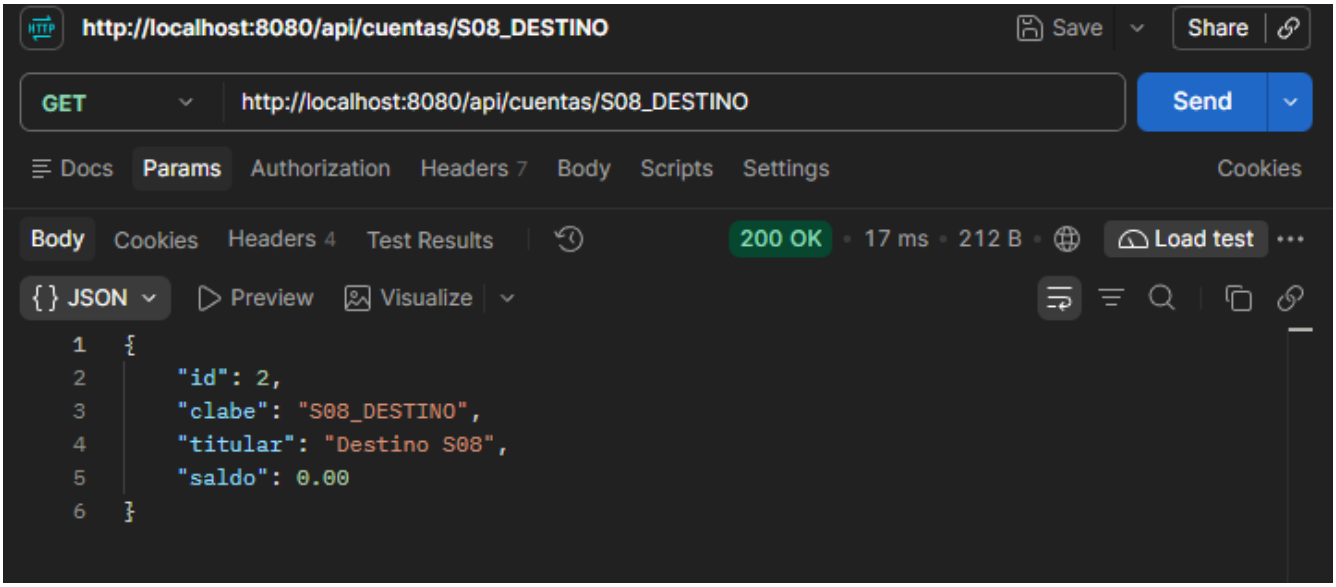



Figure 4: Saldo de la cuenta destino tras el fallo no controlado ($0.00).

El problema ocurrió porque el microservicio de transferencias hizo el cargo en la cuenta origen de forma exitosa, pero el servicio de cuentas se apagó antes de poder procesar el abono correspondiente al destino. Al no existir un mecanismo de saga o de compensación automática, la transacción se quedó a medias y el dinero ($500.00) desapareció del flujo, rompiendo la integridad del sistema.

# Parte 1 · La saga orquestada (obligatoria)

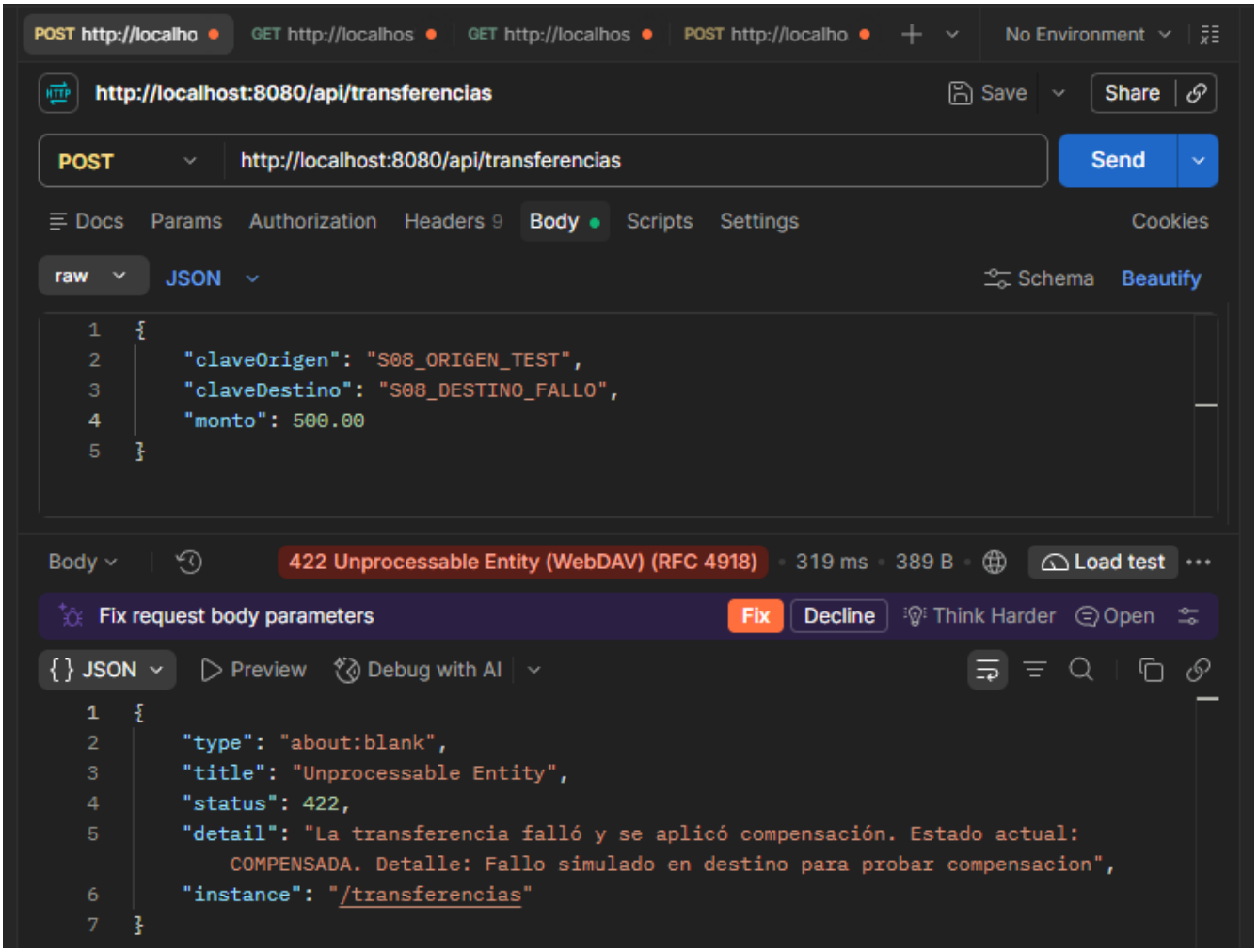



Figure 5: Respuesta controlada del servidor tras simular una falla lógica en el destino.

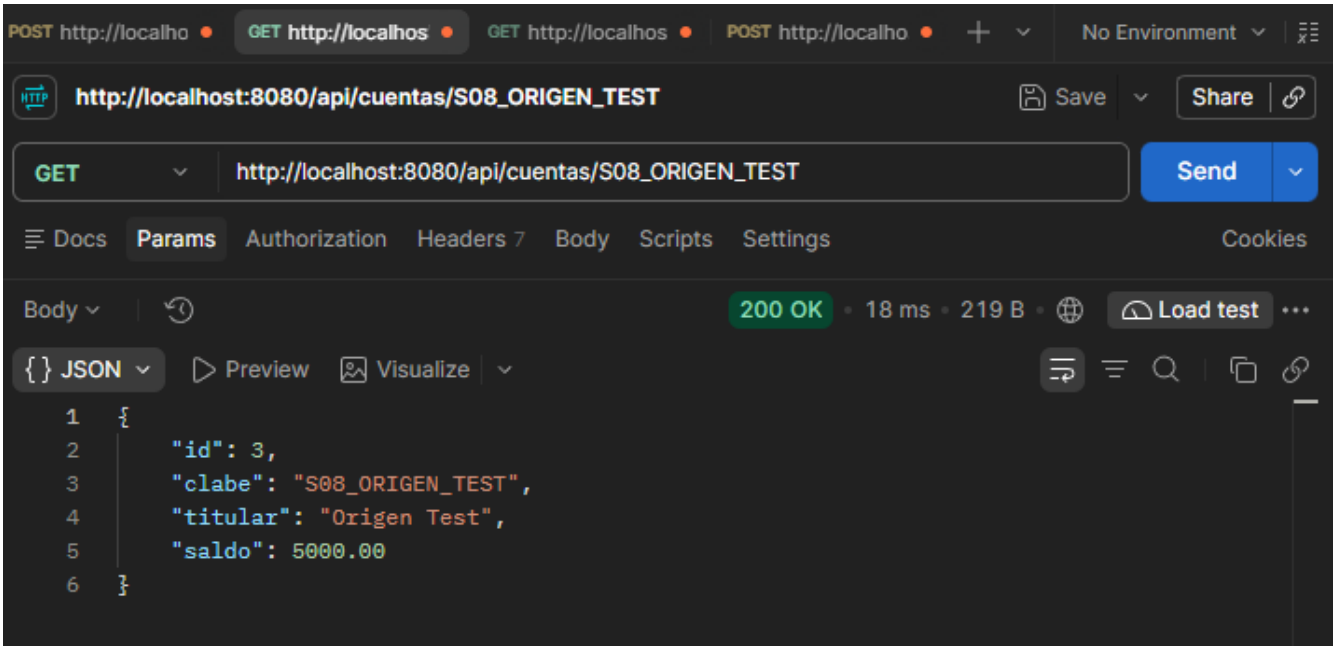



Figure 6: Restauración del saldo en la cuenta origen a $5000.00 gracias a la compensación.

Para cumplir el requerimiento de provocar la falla del abono sin apagar la infraestructura, modifiqué la clase `TransferenciaService.java` agregando una condición en el código que lanza una excepción de tipo `RuntimeException` explícita si la CLABE de destino coincide con `S08_DESTINO_FALLO`. Al ejecutar la transferencia hacia esta cuenta, el orquestador descontó $500.00 del origen, capturó inmediatamente la excepción en su intento de abono, y activó la saga de compensación revirtiendo el cargo inicial. La transferencia evolucionó correctamente al estado `COMPENSADA` y el saldo en la cuenta origen se restauró íntegramente a $5000.00.

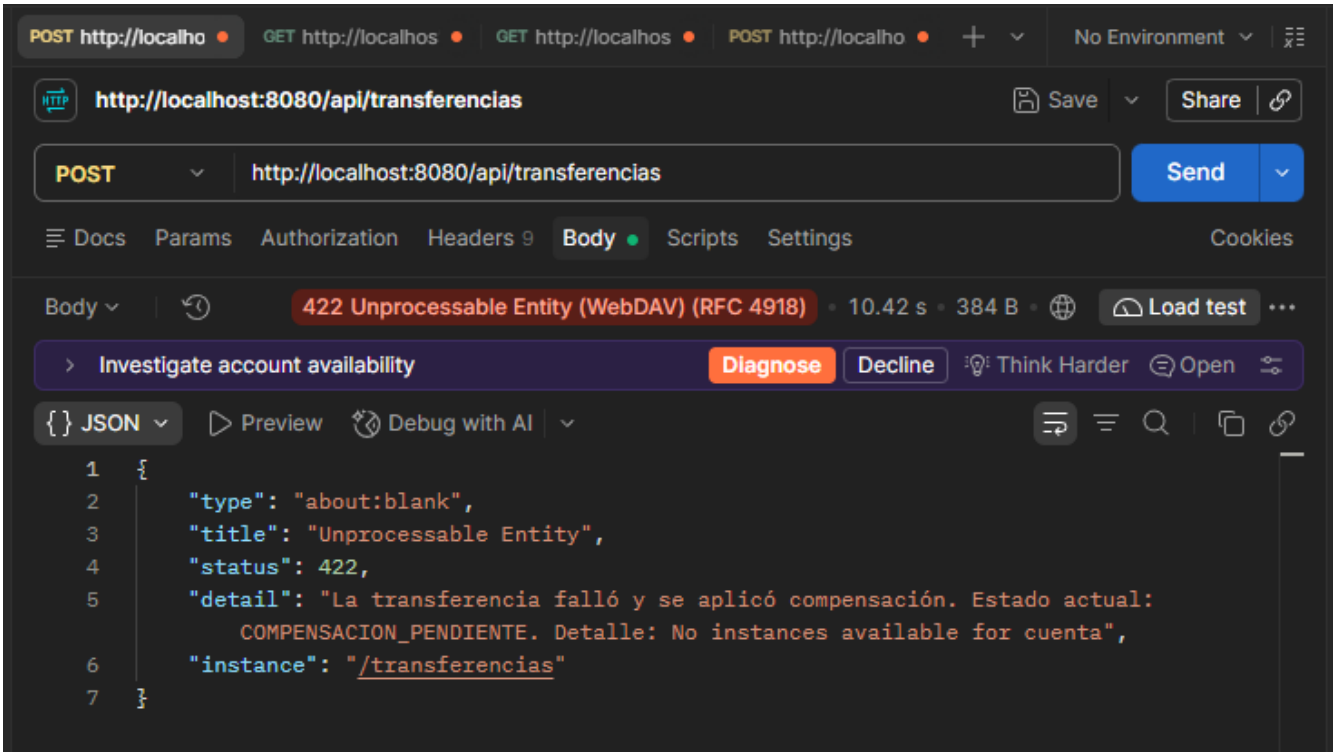



Figure 7: Error detectado mostrando el estado de COMPENSACION_PENDIENTE.

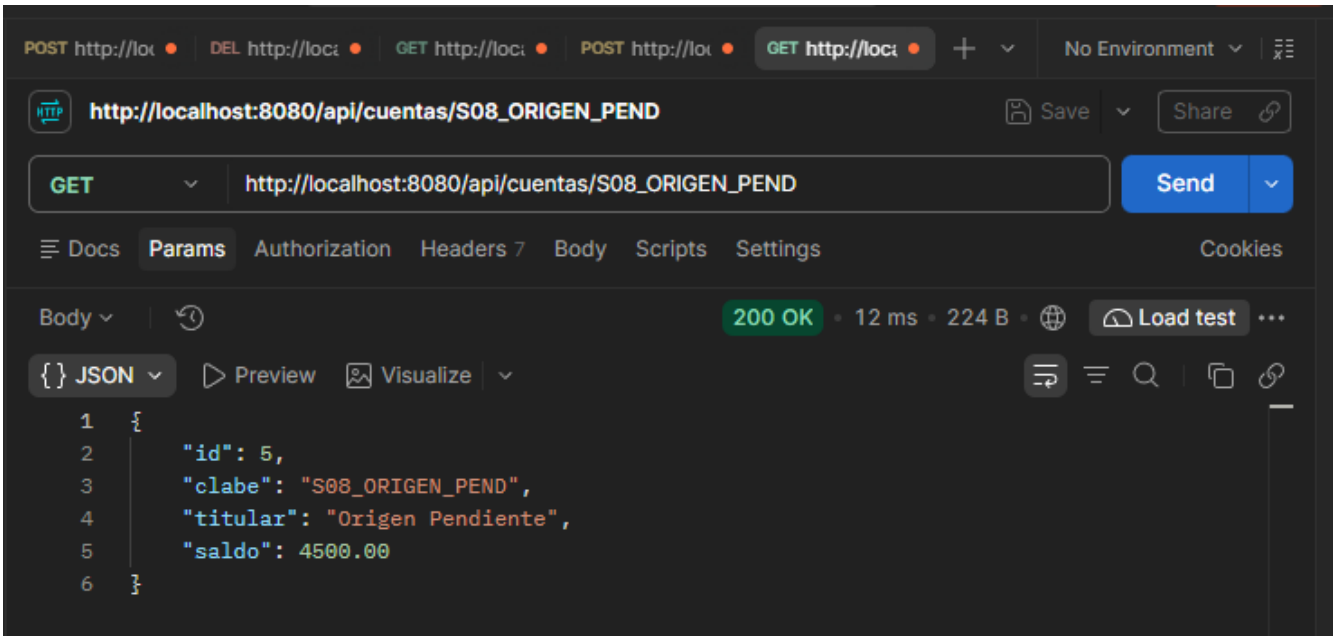



Figure 8: Saldo temporalmente descontado ($4500.00) por fallo total de infraestructura.

Para concluir y dar cierre a las transferencias atascadas en estado `COMPENSACION_PENDIENTE`, se implementaría un proceso automatizado en segundo plano (como un *Background Worker* o *CronJob* en Spring mediante la anotación `@Scheduled`). Este demonio leería periódicamente la base de datos buscando transferencias rezagadas en este estado específico, y reintentaría la llamada de abono (reembolso) hacia

la cuenta origen hasta que el microservicio de cuentas vuelva a estar disponible y responda exitosamente.

**¿Por qué la compensación debe ser idempotente?**
La compensación debe ser estrictamente idempotente debido a que el hilo de reintentos puede enviar la petición de reembolso en repetidas ocasiones, sobre todo si ocurren fallos de red intermitentes o *timeouts*. Si la operación no estuviera diseñada para ser idempotente, un reintento duplicado que pase por error a través de la red podría abonar los fondos al origen más de una vez, generando dinero de la nada e invalidando la consistencia contable de la institución bancaria.

## Parte 2 · Circuit breaker en las llamadas a cuenta

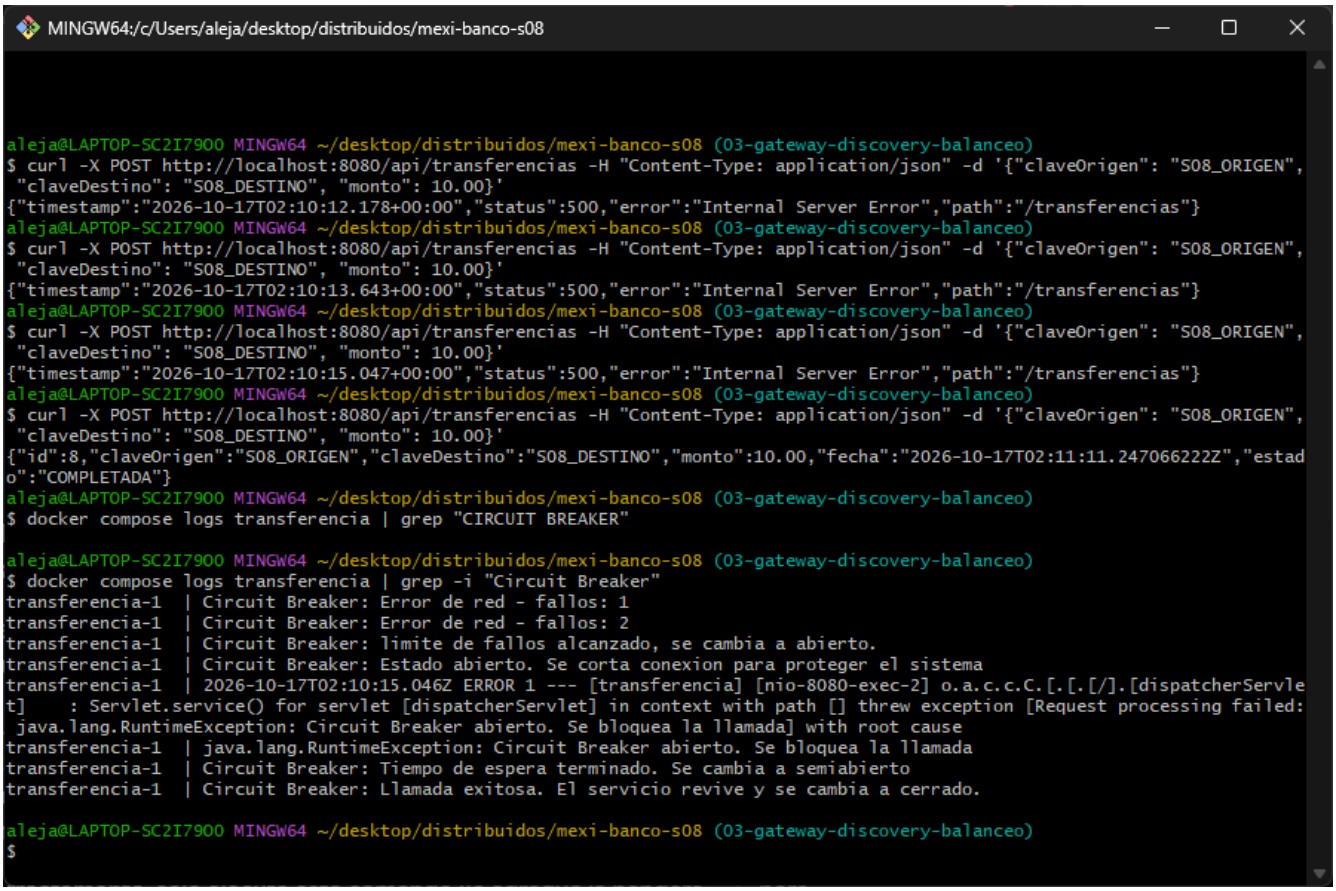



Figure 9: Comportamiento en consola de los estados CERRADO, ABIERTO y SEMIABIERTO.

Para este requerimiento, desarrollé e implementé una solución de *Circuit Breaker* programada de forma manual que protege las llamadas HTTP al microservicio de cuentas.

Como se documenta en los *logs* de la terminal, ante la caída repentina del servicio de cuentas, el circuito registra el fallo. Tras acumular dos fallos consecutivos, cambia de estado CERRADO a ABIERTO. En este estado protector, el sistema corta la conexión de manera proactiva, rechazando peticiones rápidamente sin desgastar recursos ni esperar *timeouts*. Una vez concluido el margen de tiempo configurado (10 segundos), transita a estado SEMIABIERTO para permitir una petición de prueba. Al restablecerse el contenedor

de cuentas y lograr una comunicación exitosa, el circuito asume que el servicio ”revivió” y regresa a su estado CERRADO original, restableciendo el tráfico normal.

## Parte 3 · Lo que la saga no te da (Aislamiento)

A diferencia de una transacción monolítica tradicional basada en el modelo ACID, la saga orquestada en un entorno de microservicios carece de una garantía estricta de aislamiento (*Isolation*).

Si un cliente decide consultar el saldo de su cuenta de origen exactamente en el breve intervalo de tiempo en el que se acaba de ejecutar el cargo, pero antes de que se concluya el abono (o se ejecute su debida compensación), el sistema le devolverá el saldo reducido. Este fenómeno evidencia que los estados intermedios e incompletos de la transacción son totalmente visibles para los clientes externos. Aunque la cuenta recuperará sus fondos un milisegundo después mediante el reembolso automático si algo sale mal, el cliente percibirá momentáneamente una reducción de dinero, lo cual es consecuencia directa del modelo de ”consistencia eventual” (*Eventual Consistency*) típico de la arquitectura distribuida.

# Correcciones aplicadas

## 1. Evidencia de Estado Final en Base de Datos

Para comprobar que el orquestador de la saga asienta correctamente los estados resultantes en su sistema de almacenamiento, se ejecutó una consulta SQL sobre el contenedor de PostgreSQL perteneciente a la tabla `transferencia`.

La captura a continuación indica la actualización exitosa de los registros en la base de datos, evidenciando transferencias asentadas en los estados críticos solicitados, destacando los estados `COMPENSADA` (id 2) originada por el fallo simulado en destino, y `COMPENSACION_PENDIENTE` (ids 1, 3, 4, 5, 6) derivada de la simulación del colapso total de la red durante el proceso de reembolso:

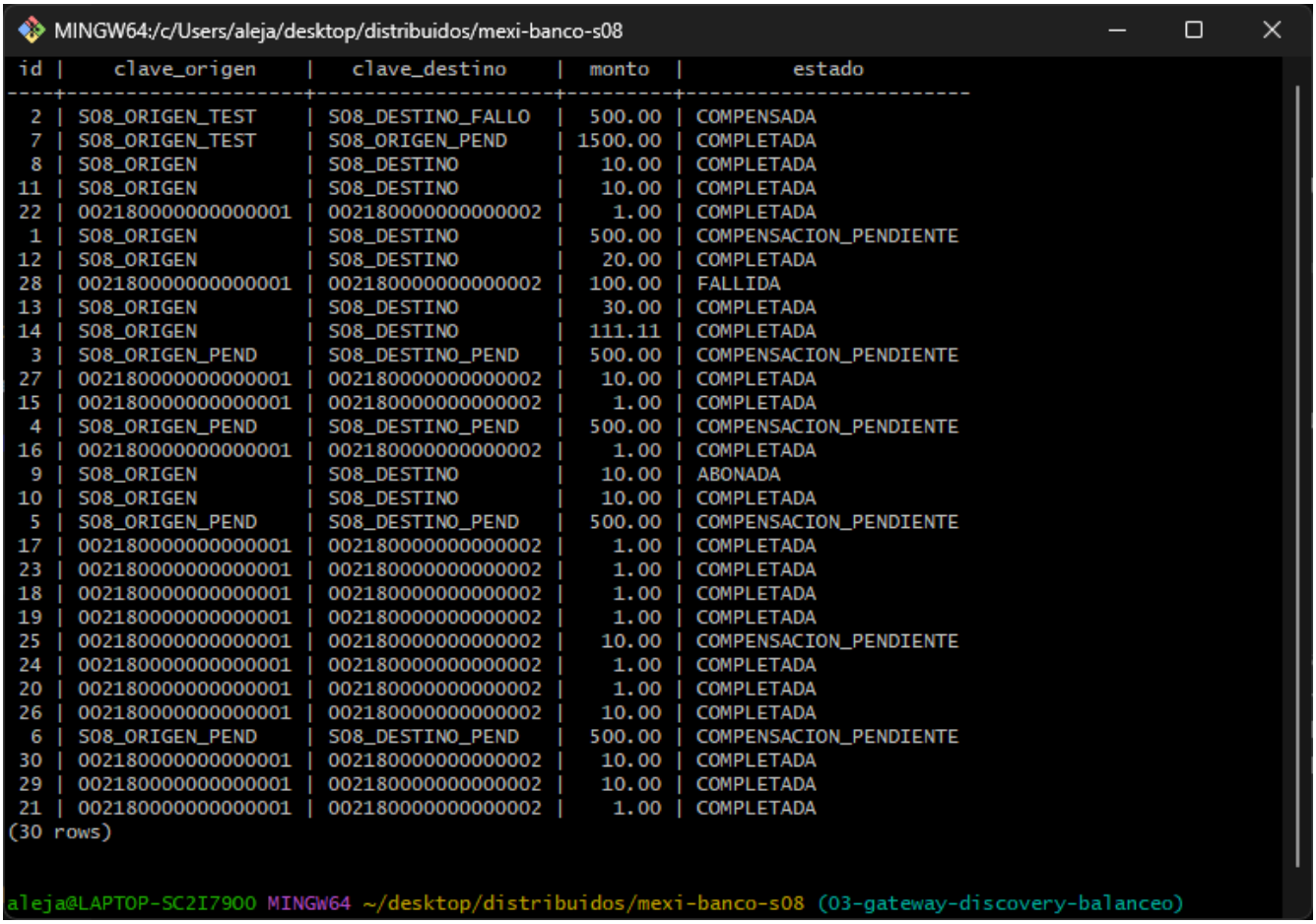



Figura 1: Consulta a la tabla de transferencias en PostgreSQL demostrando los estados asíncronos y finales orquestados por la saga.

## 2. Sección de Código Clave (Saga y Circuit Breaker)

A continuación, presento y explico los fragmentos clave que se modificaron para orquestar la saga en el servicio de transferencias.

### Orquestación de la Saga (`TransferenciaService.java`)

En lugar de que el sistema falle silenciosamente o de manera catastrófica, la transferencia guarda su progreso paso a paso. Se implementaron los siguientes cambios:

```
try {
    // 1. Cargo al origen y avance de estado
    origen = circuitBreaker.proteger(() -> cuentas.cargar(claveOrigen, monto));
    transferencia.setEstado("CARGADA");
    transferencias.save(transferencia);

    // 2. Simulacion de falla en destino (Reto S08)
    if ("S08_DESTINO_FALLO".equals(claveDestino)) {
        throw new RuntimeException("Fallo simulado en destino para probar compensacion");
    }

    // 3. Abono al destino (Solo se alcanza si no hay fallos)
    destino = circuitBreaker.proteger(() -> cuentas.abonar(claveDestino, monto));
    transferencia.setEstado("COMPLETADA");
    transferencias.save(transferencia);

} catch (Exception e) {
    // 4. Bloque de Compensacion
    if ("CARGADA".equals(transferencia.getEstado())) {
        try {
            circuitBreaker.proteger(() -> { cuentas.abonar(claveOrigen, monto); return null; });
            transferencia.setEstado("COMPENSADA");
        } catch (Exception exCompensacion) {
            transferencia.setEstado("COMPENSACION_PENDIENTE");
        }
    } else {
        transferencia.setEstado("FALLIDA");
    }
    transferencias.save(transferencia);
    throw new ReglaDeNegocioException("Transferencia fallo. Estado: " + transferencia.getEstado() + ". Detalle: " + e.getMessage());
}
```

Listing 1: Simulación de falla y captura para compensación en la Saga

El orquestador es el único responsable de llevar el control de la operación. En el bloque principal de ejecución, la transacción comienza ejecutando el cargo y actualizando su estado a `CARGADA` en la base de datos. Si durante el siguiente intento de abono, o de manera específica en la simulación de falla con la cuenta destino, ocurre un error en la red, el bloque controlador de excepciones toma el mando operativo.

Dado que el estado guardado previamente era `CARGADA`, el orquestador tiene plena consciencia de que el dinero ya abandonó el origen. Por consiguiente, ejecuta una **operación de compensación** que consiste en reembolsar dicho monto a la cuenta de origen. Si este reembolso finaliza con éxito, el estado definitivo pasa a ser `COMPENSADA`; si por el contrario la red también falla durante la compensación, se marca cautelarmente

como `COMPENSACION_PENDIENTE` para ser reintentada a futuro, garantizando que el capital jamás quede perdido en un limbo transaccional.

### Implementación manual del Circuit Breaker

Para proteger el sistema de saturaciones, implementé una clase controladora, la cual envuelve las llamadas asíncronas que hace el servicio de transferencias hacia el de cuentas:

```
public synchronized <T> T proteger(Supplier<T> accion) {
    if (estado == Estado.ABIERTO) {
        if (System.currentTimeMillis() - tiempoApertura > ESPERA_MS) {
            System.out.println("Circuit Breaker: Tiempo de espera terminado. Se cambia a semiabierto");
            estado = Estado.SEMIABIERTO;
        } else {
            System.out.println("Circuit Breaker: Estado abierto. Se corta conexion para proteger el sistema");
            throw new RuntimeException("Circuit Breaker abierto. Se bloquea la llamada");
        }
    }

    try {
        T resultado = accion.get();
        if (estado == Estado.SEMIABIERTO) {
            System.out.println("Circuit Breaker: Llamada exitosa. El servicio revive y se cambia a cerrado.");
            estado = Estado.CERRADO;
            fallos = 0;
        }
        return resultado;
    } catch (Exception e) {
        fallos++;
        System.out.println("Circuit Breaker: Error de red - fallos: " + fallos);
        if (estado == Estado.SEMIABIERTO || fallos >= 2) {
            System.out.println("Circuit Breaker: limite de fallos alcanzado, se cambia a abierto.");
            estado = Estado.ABIERTO;
            tiempoApertura = System.currentTimeMillis();
        }
        throw e;
    }
}
```

Listing 2: Lógica de protección en Circuit Breaker.java

La clase evalúa sus tres estados fundamentales. Cuando permanece en estado cerrado, intenta ejecutar la acción HTTP de red solicitada. Si dicha red se interrumpe y ocurre una excepción, captura el evento incrementando un contador de fallas de manera silenciosa. Al acumular un límite definido, en este caso dos fallos consecutivos, el algoritmo rompe el circuito cambiando a estado abierto e inicia un cronómetro interno.

Bajo el estado abierto, cualquier petición subsecuente es rechazada inmediatamente ahorrando tiempo, latencia y recursos computacionales que se habrían malgastado en repetidos y fútiles intentos por contactar a un servicio fuera de línea. Una vez transcurridos 10 segundos, el circuito entra en estado de prueba semiabierto. Si la siguiente petición de prueba triunfa, el contador se reinicia y el circuito se cierra de nuevo normalizando las operaciones; si vuelve a fallar, regresa a un estado abierto reanudando la cuenta de espera protectora.

## 3. Diagrama de Estados de la Transferencia (Saga)

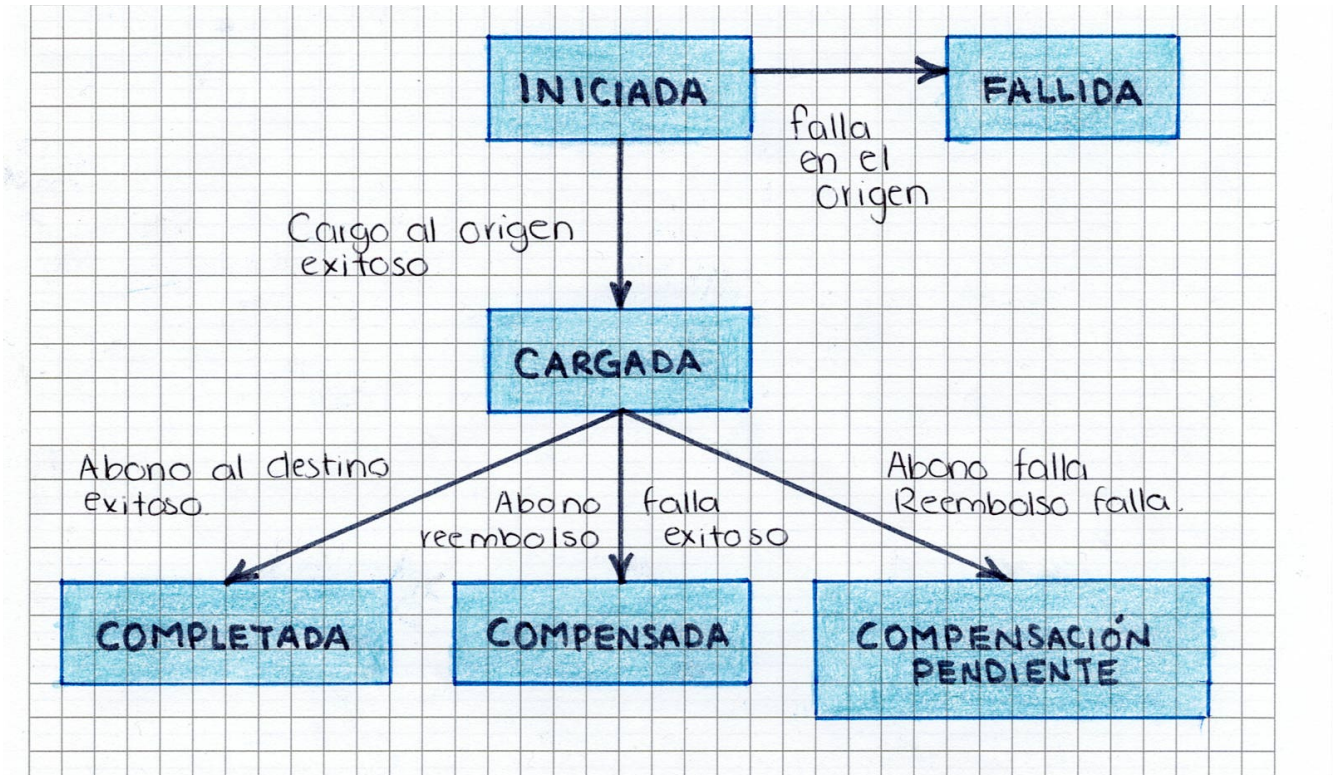



Figura 2: Diagrama de estados de la Transferencia (Saga)

- **INICIADA:** La transferencia ha sido registrada en el sistema de manera formal, previa a cualquier interacción con los saldos de las cuentas involucradas.
- **FALLIDA:** El intento inicial de descontar los fondos de la cuenta de origen fracasó, cancelando la operación de manera inmediata sin requerir un proceso de compensación.
- **CARGADA:** Los fondos fueron retirados exitosamente de la cuenta de origen y se encuentran en un tránsito lógico a la espera de ser depositados.
- **COMPLETADA:** El capital se depositó de manera exitosa en la cuenta de destino, culminando la transacción de forma íntegra.
- **COMPENSADA:** Surgió un error al intentar transferir los fondos al destino, provocando que el sistema ejecutara un reembolso exitoso hacia la cuenta de origen.

- **COMPENSACION_PENDIENTE:** El sistema fracasó al intentar realizar el reembolso debido a problemas de red, dejando la transacción en un estado de alerta para forzar un reintento a futuro y evitar la pérdida del capital.